

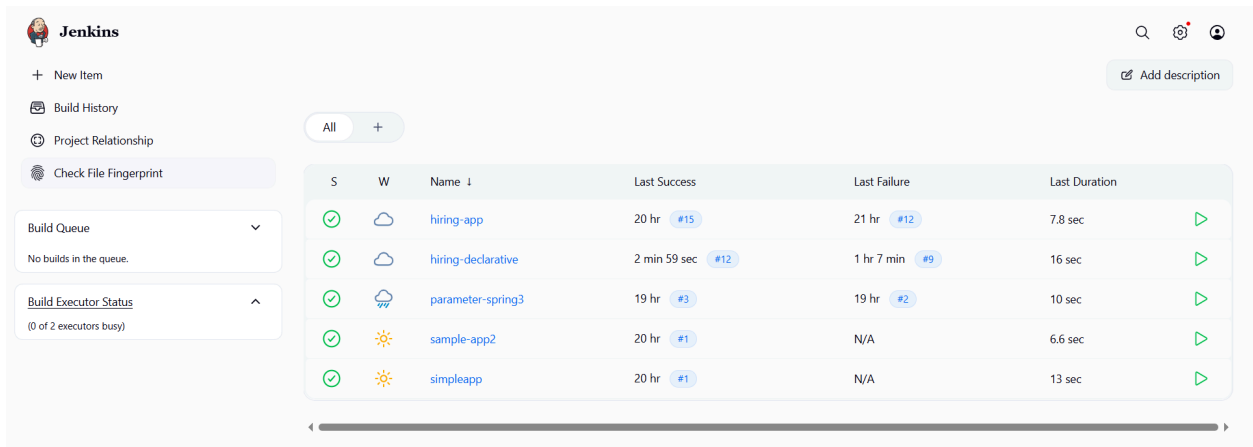
# Jenkins challenge

Use the below code and implement our CICD stages.

- Source Code:<https://github.com/betawins/hiring-app.git>
  - Repository contains the application code for the hiring app.
- Create one Declarative pipeline job
- Create one Scripted pipeline job
- Create one multi stage pipeline job
- Create one parallel stage pipeline job

**Declarative pipeline job:**

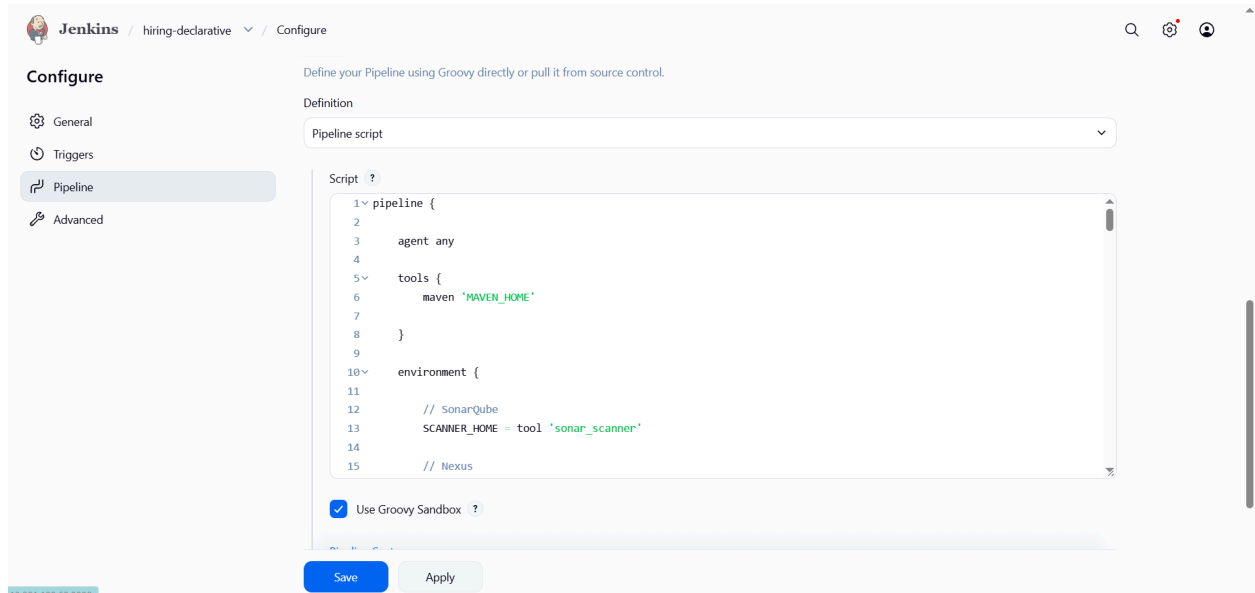
**Create new item→select pipeline→save**



The screenshot shows the Jenkins dashboard with a sidebar on the left containing navigation links: '+ New Item', 'Build History', 'Project Relationship', and 'Check File Fingerprint'. The main area displays a table of jobs with columns for status (S), icon (W), name, last success, last failure, and last duration. The jobs listed are 'hiring-app', 'hiring-declarative', 'parameter-spring3', 'sample-app2', and 'simpleapp'. All jobs show a green checkmark in the status column, indicating successful builds. The 'hiring-app' job has a last success of 20 hr #15 and a last failure of 21 hr #12. The 'hiring-declarative' job has a last success of 2 min 59 sec #12 and a last failure of 1 hr 7 min #9. The 'parameter-spring3' job has a last success of 19 hr #3 and a last failure of 19 hr #2. The 'sample-app2' job has a last success of 20 hr #1 and no last failure. The 'simpleapp' job has a last success of 20 hr #1 and no last failure. The 'hiring-app' job has a last duration of 7.8 sec, 'hiring-declarative' has 16 sec, 'parameter-spring3' has 10 sec, 'sample-app2' has 6.6 sec, and 'simpleapp' has 13 sec. A search bar and an 'Add description' button are visible in the top right corner.

| S | W | Name               | Last Success     | Last Failure  | Last Duration |
|---|---|--------------------|------------------|---------------|---------------|
| ✓ | ☁ | hiring-app         | 20 hr #15        | 21 hr #12     | 7.8 sec       |
| ✓ | ☁ | hiring-declarative | 2 min 59 sec #12 | 1 hr 7 min #9 | 16 sec        |
| ✓ | ☁ | parameter-spring3  | 19 hr #3         | 19 hr #2      | 10 sec        |
| ✓ | ☀ | sample-app2        | 20 hr #1         | N/A           | 6.6 sec       |
| ✓ | ☀ | simpleapp          | 20 hr #1         | N/A           | 13 sec        |

Open job→ click on configure→write pipeline→save& build



**pipeline {**

**agent any**

**tools {**  
**maven 'MAVEN\_HOME'**  
**}**

**environment {**  
  
**// SonarQube**  
**SCANNER\_HOME = tool 'sonar\_scanner'**  
  
**// Nexus**  
**NEXUS\_VERSION = "nexus3"**  
**NEXUS\_PROTOCOL = "http"**  
**NEXUS\_URL = "13.232.205.190:8081"**  
**NEXUS\_REPOSITORY = "hiring-app"**  
**NEXUS\_CREDENTIAL\_ID = "nexus"**  
  
**// Artifact Details**  
**GROUP\_ID = "in.javahome"**

```
ARTIFACT_ID = "hiring"  
VERSION = "1.0.${BUILD_NUMBER}-SNAPSHOT"
```

```
// Tomcat  
TOMCAT_URL = "http://13.201.30.120:8080/"  
TOMCAT_CREDENTIAL_ID = "tomcat"
```

```
// Slack  
SLACK_CHANNEL = "jobs"
```

```
}
```

```
stages {
```

```
    stage('Clean Workspace') {  
        steps {  
            cleanWs()  
        }  
    }  
}
```

```
    stage('Clone Source Code') {  
        steps {  
  
            git branch: 'main',  
            url: 'https://github.com/betawins/hiring-app.git'  
        }  
    }  
}
```

```
    stage('Build Application') {  
        steps {  
  
            sh "${tool 'MAVEN_HOME'}/bin/mvn clean install"  
        }  
    }  
}
```

```
    stage('Run Unit Tests') {  
        steps {  
  
            sh 'mvn test'  
        }  
    }  
}
```

```

stage('SonarQube Code Analysis') {

    steps {

        withSonarQubeEnv('sonarqube') {

            sh """
            ${SCANNER_HOME}/bin/sonar-scanner \
            -Dsonar.projectKey=hiring-app \
            -Dsonar.projectName=hiring-app \
            -Dsonar.sources=.
            """
        }
    }
}

```

```

stage('Package Application') {

    steps {

        sh 'mvn clean package'
    }
}

```

```

stage('Archive Artifact') {

    steps {

        archiveArtifacts artifacts: 'target/hiring.war'
    }
}

```

```

stage('Upload WAR to Nexus') {

    steps {

        nexusArtifactUploader(

            nexusVersion: NEXUS_VERSION,

```

```

    protocol: NEXUS_PROTOCOL,
    nexusUrl: NEXUS_URL,
    repository: NEXUS_REPOSITORY,
    credentialsId: NEXUS_CREDENTIAL_ID,
    groupId: GROUP_ID,
    version: VERSION,

    artifacts: [
        [
            artifactId: ARTIFACT_ID,
            classifier: "",
            file: 'target/hiring.war',
            type: 'war'
        ]
    ]
    )
}
}

stage('Deploy to Tomcat') {

    steps {

        deploy adapters: [
            tomcat9(
                credentialsId: TOMCAT_CREDENTIAL_ID,
                path: "",
                url: "${TOMCAT_URL}"
            )
        ],

        contextPath: 'hiring-app',
        war: 'target/hiring.war'
    }
}

post {

    success {

```

```

        slackSend(
            channel: "${SLACK_CHANNEL}",
            color: 'good',
            message: """"
SUCCESS:
Job Name: ${env.JOB_NAME}
Build Number: ${env.BUILD_NUMBER}
Build URL: ${env.BUILD_URL}
""""
        )
    }

    failure {

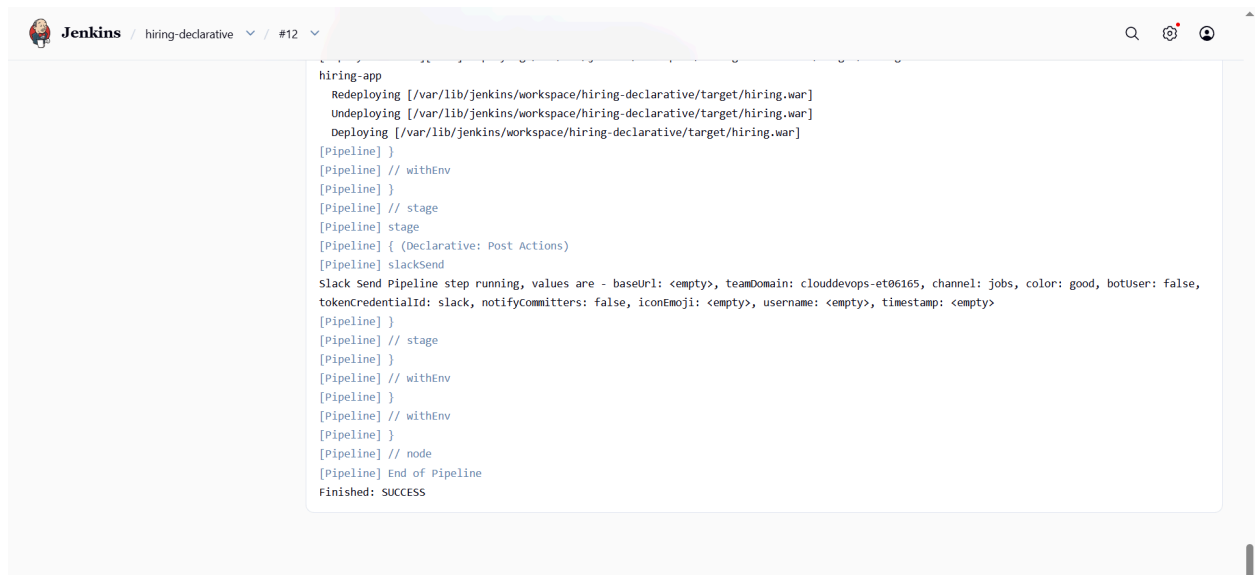
        slackSend(
            channel: "${SLACK_CHANNEL}",
            color: 'danger',
            message: """"
FAILED:
Job Name: ${env.JOB_NAME}
Build Number: ${env.BUILD_NUMBER}
Build URL: ${env.BUILD_URL}
""""
        )
    }
}
}
}

```

## Build successfully

The screenshot displays the Jenkins web interface for a pipeline named 'hiring-declarative'. The interface includes a left-hand navigation menu with options like 'Status', 'Changes', 'Build Now', 'Configure', 'Delete Pipeline', 'SonarQube', 'Stages', 'Rename', and 'Pipeline Syntax'. The main content area shows the build status as 'hiring-declarative' with a green checkmark. Below this, there's a section for 'Last Successful Artifacts' showing 'hiring.war' (1.66 KiB) with a 'view' link. A 'SonarQube Quality Gate' section indicates 'hiring-app' has 'Passed' and 'server-side processing' is a 'Success'. A 'Permalinks' section lists various build links, including 'Last build (#12), 14 min ago', 'Last stable build (#12), 14 min ago', 'Last successful build (#12), 14 min ago', 'Last failed build (#9), 1 hr 19 min ago', 'Last unsuccessful build (#9), 1 hr 19 min ago', and 'Last completed build (#12), 14 min ago'. At the bottom, there's a 'Builds' section with a search filter and a list of builds.

## Check console output:



```
hiring-app
  Redeploying [/var/lib/jenkins/workspace/hiring-declarative/target/hiring.war]
  Undeploying [/var/lib/jenkins/workspace/hiring-declarative/target/hiring.war]
  Deploying [/var/lib/jenkins/workspace/hiring-declarative/target/hiring.war]
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: clouddevops-et06165, channel: jobs, color: good, botuser: false,
tokenCredentialId: slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

## Slack notification

added an integration to this channel. jenkins



jenkins APP 3:46 PM

Slack/Jenkins plugin: you're all set on <http://13.233.253.54:8080/>

SUCCESS:

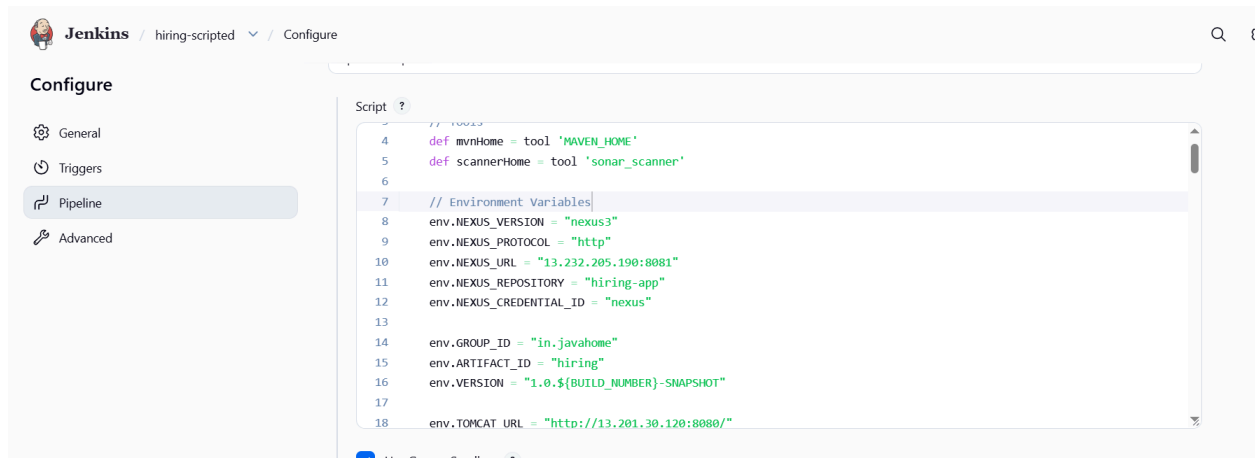
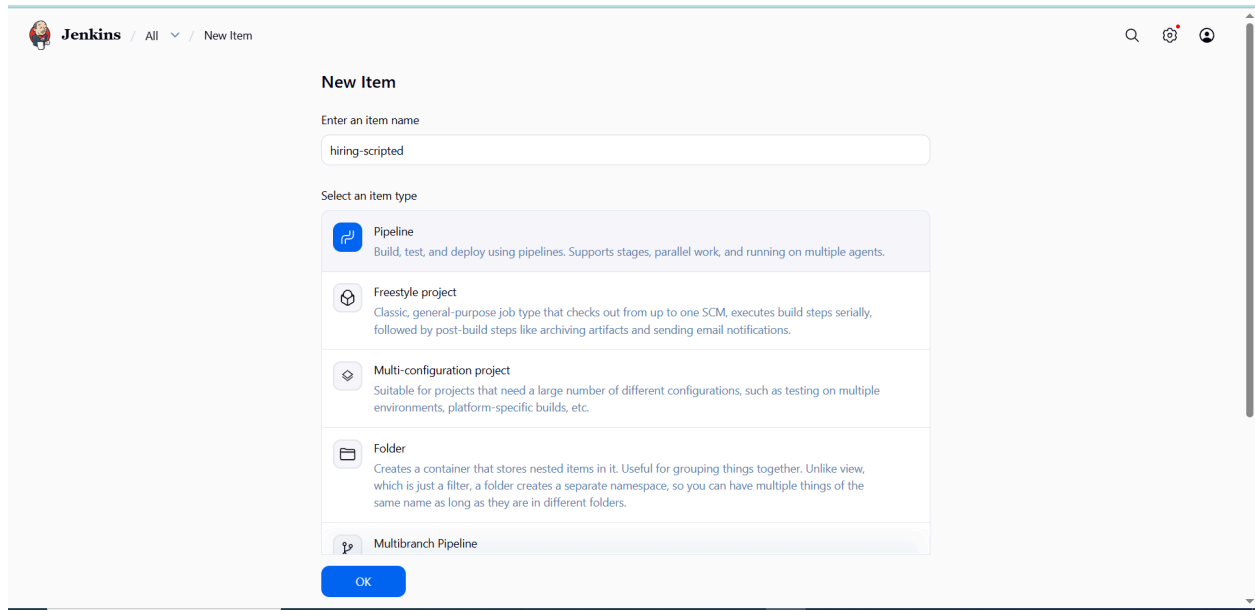
Job Name: hiring-declarative

Build Number: 12

Build URL: <http://13.233.253.54:8080/job/hiring-declarative/12/>

# Scripted pipeline job:

Create new item → select pipeline → configure → select scripted pipeline



```
node {
```

```
    // Tools
```

```
    def mvnHome = tool 'MAVEN_HOME'
```



```
def scannerHome = tool 'sonar_scanner'

// Environment Variables
env.NEXUS_VERSION = "nexus3"
env.NEXUS_PROTOCOL = "http"
env.NEXUS_URL = "13.232.205.190:8081"
env.NEXUS_REPOSITORY = "hiring-app"
env.NEXUS_CREDENTIAL_ID = "nexus"

env.GROUP_ID = "in.javahome"
env.ARTIFACT_ID = "hiring"
env.VERSION = "1.0.${BUILD_NUMBER}-SNAPSHOT"

env.TOMCAT_URL = "http://13.201.30.120:8080/"
env.TOMCAT_CREDENTIAL_ID = "tomcat"

env.SLACK_CHANNEL = "jobs"

try {

    stage('Clean Workspace') {

        cleanWs()
    }

    stage('Clone Source Code') {

        git branch: 'main',
        url: 'https://github.com/betawins/hiring-app.git'
    }
}
```

```
stage('Build Application') {

    sh "${mvnHome}/bin/mvn clean install"
}

stage('Run Unit Tests') {

    sh "${mvnHome}/bin/mvn test"
}

stage('SonarQube Code Analysis') {

    withSonarQubeEnv('sonarqube') {

        sh """
        ${scannerHome}/bin/sonar-scanner \
        -Dsonar.projectKey=hiring-app \
        -Dsonar.projectName=hiring-app \
        -Dsonar.sources=.
        """
    }
}

stage('Package Application') {

    sh "${mvnHome}/bin/mvn clean package"
}

stage('Archive Artifact') {

    archiveArtifacts artifacts: 'target/hiring.war'
```

```
}
```

```
stage('Upload WAR to Nexus') {
```

```
    nexusArtifactUploader(
```

```
        nexusVersion: env.NEXUS_VERSION,
```

```
        protocol: env.NEXUS_PROTOCOL,
```

```
        nexusUrl: env.NEXUS_URL,
```

```
        repository: env.NEXUS_REPOSITORY,
```

```
        credentialsId: env.NEXUS_CREDENTIAL_ID,
```

```
        groupId: env.GROUP_ID,
```

```
        version: env.VERSION,
```

```
        artifacts: [
```

```
            [
```

```
                artifactId: env.ARTIFACT_ID,
```

```
                classifier: "",
```

```
                file: 'target/hiring.war',
```

```
                type: 'war'
```

```
            ]
```

```
        ]
```

```
    )
```

```
}
```

```
stage('Deploy to Tomcat') {
```

```
    deploy adapters: [
```

```
        tomcat9(
```

```
            credentialsId: env.TOMCAT_CREDENTIAL_ID,
```

```
            path: "",
```

```
        url: env.TOMCAT_URL
    )
],
```

```
    contextPath: 'hiring-app',
    war: 'target/hiring.war'
}
```

```
stage('Slack Notification Success') {
```

```
    slackSend(
        channel: env.SLACK_CHANNEL,
        color: 'good',
        tokenCredentialId: 'slack',
        message: ""
```

```
SUCCESS:
```

```
Job Name: ${env.JOB_NAME}
```

```
Build Number: ${env.BUILD_NUMBER}
```

```
Build URL: ${env.BUILD_URL}
```

```
""
```

```
    )
}
```

```
} catch (Exception e) {
```

```
stage('Slack Notification Failure') {
```

```
    slackSend(
        channel: env.SLACK_CHANNEL,
        color: 'danger',
        tokenCredentialId: 'slack',
```

message: ""

FAILED:

Job Name: \${env.JOB\_NAME}

Build Number: \${env.BUILD\_NUMBER}

Build URL: \${env.BUILD\_URL}

""

)

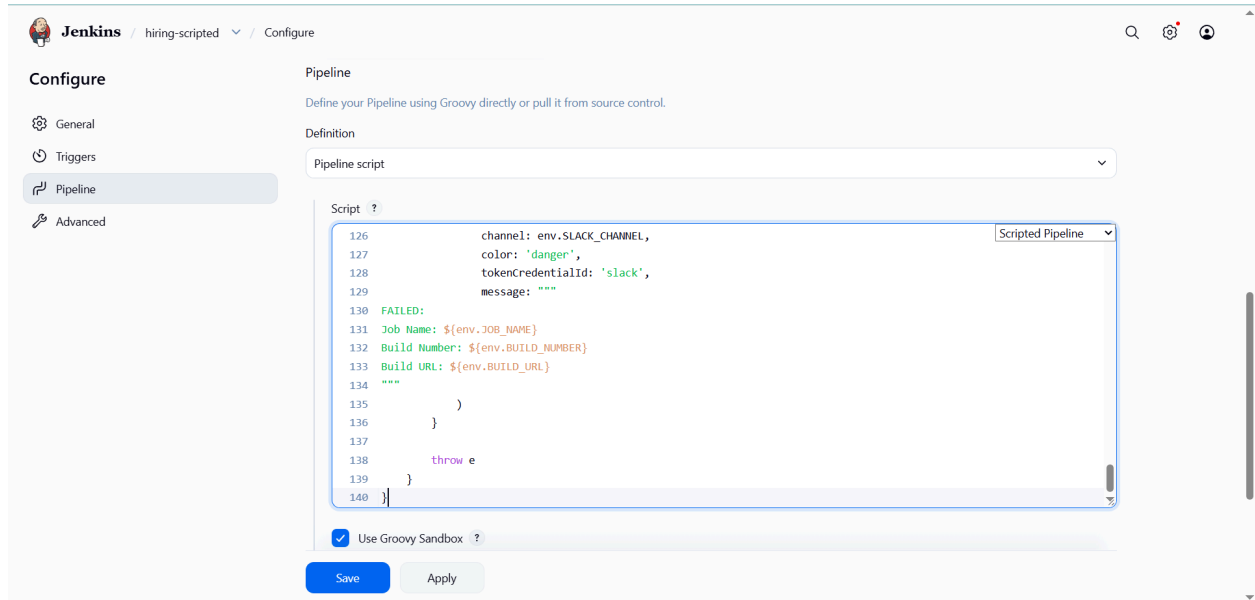
}

throw e

}

}

# save&build

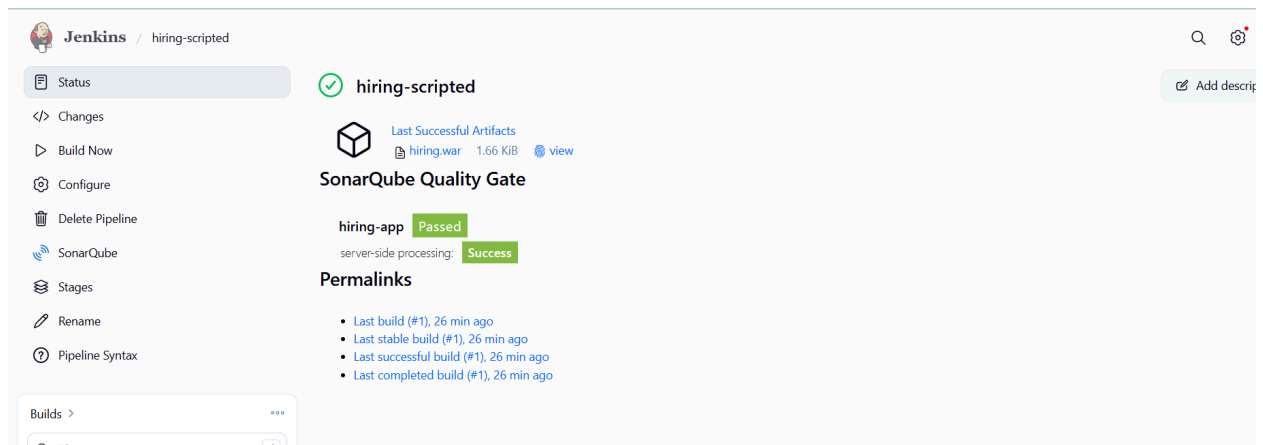


The screenshot shows the Jenkins configuration page for a pipeline named 'hiring-scripted'. The left sidebar contains a 'Configure' menu with options: General, Triggers, Pipeline (selected), and Advanced. The main area is titled 'Pipeline' and includes a description: 'Define your Pipeline using Groovy directly or pull it from source control.' Below this is a 'Definition' section with a dropdown menu set to 'Pipeline script'. A code editor displays a Groovy script for a scripted pipeline. The script includes a Slack notification step that fails if the build is not successful, and then throws an exception. The script is as follows:

```
126         channel: env.SLACK_CHANNEL,
127         color: 'danger',
128         tokenCredentialId: 'slack',
129         message: ""
130     FAILED:
131     Job Name: ${env.JOB_NAME}
132     Build Number: ${env.BUILD_NUMBER}
133     Build URL: ${env.BUILD_URL}
134     ""
135     )
136     }
137
138     throw e
139 }
140 }
```

Below the code editor, there is a checkbox labeled 'Use Groovy Sandbox' which is checked. At the bottom of the configuration area are 'Save' and 'Apply' buttons.

# Build successfull

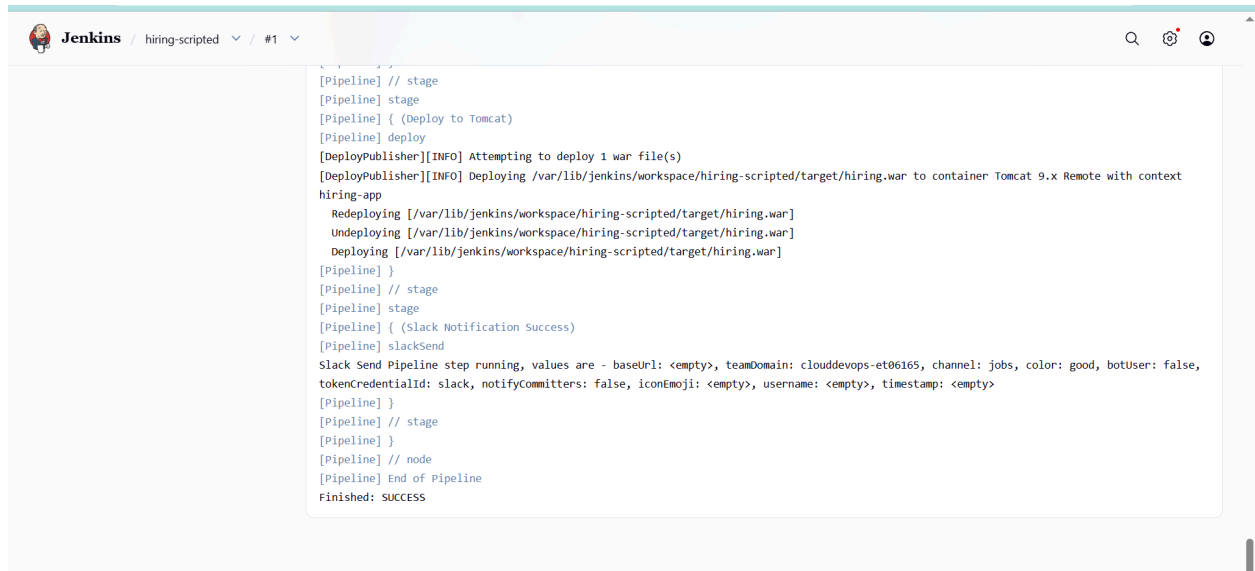


The screenshot shows the Jenkins build page for the 'hiring-scripted' pipeline. The left sidebar contains a 'Status' menu with options: Status (selected), Changes, Build Now, Configure, Delete Pipeline, SonarQube, Stages, Rename, and Pipeline Syntax. The main area displays the build status as 'hiring-scripted' with a green checkmark. Below this, there is a section for 'Last Successful Artifacts' showing a file named 'hiring.war' with a size of 1.66 KiB and a 'view' button. A 'SonarQube Quality Gate' section shows the status 'Passed' for 'hiring-app' and 'Success' for 'server-side processing'. A 'Permalinks' section lists the following links:

- Last build (#1), 26 min ago
- Last stable build (#1), 26 min ago
- Last successful build (#1), 26 min ago
- Last completed build (#1), 26 min ago

At the bottom left, there is a 'Builds' section with a dropdown menu and a search bar.

## Check console output



The screenshot shows the Jenkins console output for a job named 'hiring-scripted'. The output is as follows:

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to Tomcat)
[Pipeline] deploy
[DeployPublisher][INFO] Attempting to deploy 1 war file(s)
[DeployPublisher][INFO] Deploying /var/lib/jenkins/workspace/hiring-scripted/target/hiring.war to container Tomcat 9.x Remote with context hiring-app
  Redeploying [/var/lib/jenkins/workspace/hiring-scripted/target/hiring.war]
  Undeploying [/var/lib/jenkins/workspace/hiring-scripted/target/hiring.war]
  Deploying [/var/lib/jenkins/workspace/hiring-scripted/target/hiring.war]
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Slack Notification Success)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: cloudevops-et06165, channel: jobs, color: good, botUser: false, tokenCredentialId: slack, notifyCommiters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

## Slack notification



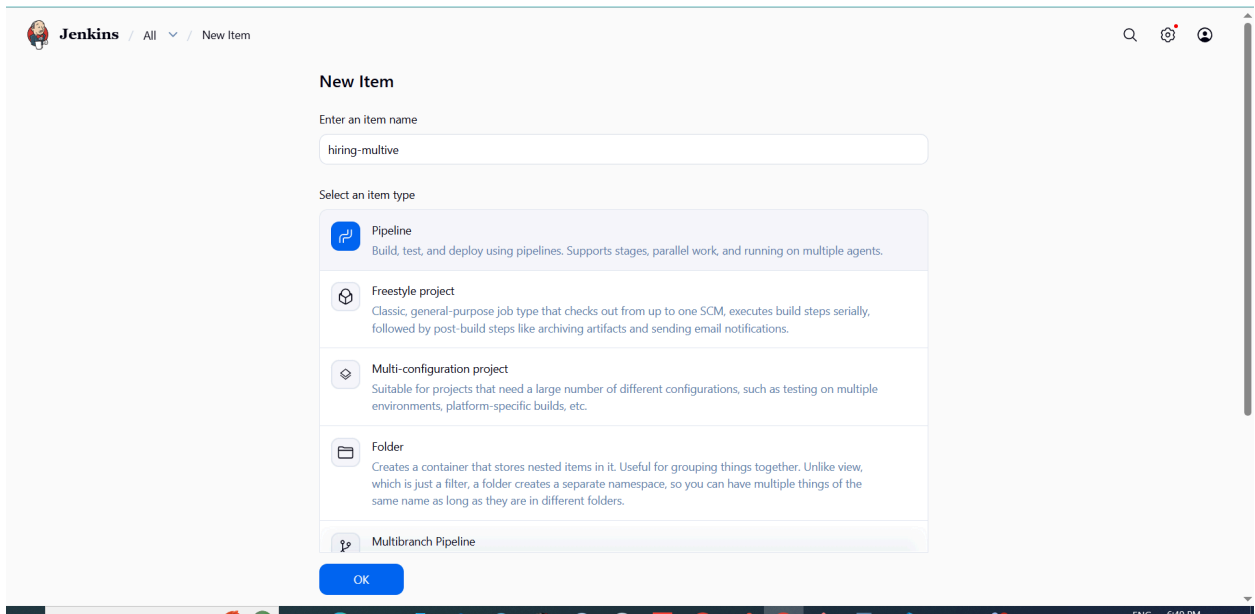
## multi stage pipeline job:

here is a proper Multi-Stage Declarative Jenkins Pipeline using:

- GitHub
- Maven
- SonarQube
- Nexus
- Tomcat
- Slack

This pipeline separates each CI/CD activity into independent stages.

## Create new item→select pipeline→configure



The screenshot shows the Jenkins 'New Item' configuration page. At the top, the breadcrumb is 'Jenkins / All / New Item'. The main heading is 'New Item'. Below it, there is a text input field for 'Enter an item name' with the value 'hiring-multive'. Underneath, a section titled 'Select an item type' lists four options: 'Pipeline' (selected), 'Freestyle project', 'Multi-configuration project', and 'Folder'. Each option has a brief description. At the bottom of the list is 'Multibranch Pipeline'. An 'OK' button is located at the bottom right of the selection area.

**New Item**

Enter an item name

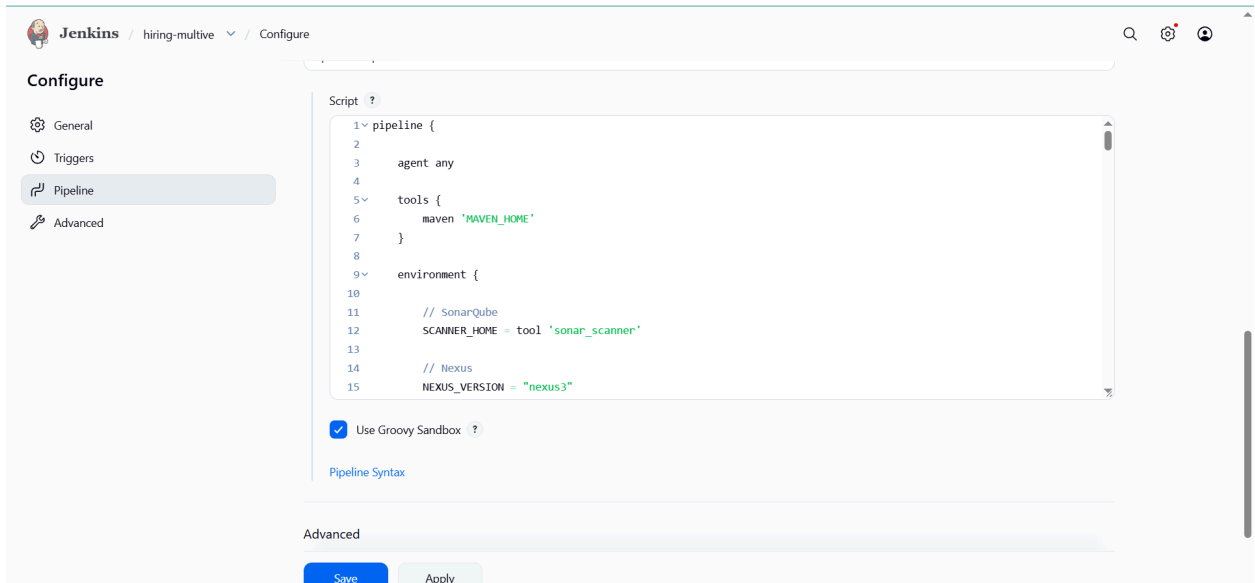
hiring-multive

Select an item type

- Pipeline**  
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**  
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project**  
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**  
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- Multibranch Pipeline**

OK

## Write pipeline



The screenshot shows the Jenkins 'Configure' page for the 'hiring-multive' item. The breadcrumb is 'Jenkins / hiring-multive / Configure'. On the left, a sidebar titled 'Configure' has four tabs: 'General', 'Triggers', 'Pipeline' (selected), and 'Advanced'. The main area is titled 'Script' and contains a text editor with a Groovy pipeline script. Below the editor, there is a checkbox for 'Use Groovy Sandbox' which is checked. At the bottom, there are 'Save' and 'Apply' buttons.

**Configure**

- General
- Triggers
- Pipeline**
- Advanced

**Script**

```
1 pipeline {
2
3   agent any
4
5   tools {
6     maven 'MAVEN_HOME'
7   }
8
9   environment {
10
11     // SonarQube
12     SCANNER_HOME = tool 'sonar_scanner'
13
14     // Nexus
15     NEXUS_VERSION = "nexus3"
```

☒ Use Groovy Sandbox

[Pipeline Syntax](#)

**Advanced**

Save Apply



```
pipeline {  
  
    agent any  
  
    tools {  
        maven 'MAVEN_HOME'  
    }  
  
    environment {  
  
        // SonarQube  
        SCANNER_HOME = tool 'sonar_scanner'  
  
        // Nexus  
        NEXUS_VERSION = "nexus3"  
        NEXUS_PROTOCOL = "http"  
        NEXUS_URL = "13.232.205.190:8081"  
        NEXUS_REPOSITORY = "hiring-app"  
        NEXUS_CREDENTIAL_ID = "nexus"  
  
        // Artifact Details  
        GROUP_ID = "in.javahome"  
        ARTIFACT_ID = "hiring"  
        VERSION = "1.0.${BUILD_NUMBER}-SNAPSHOT"  
  
        // Tomcat  
        TOMCAT_URL = "http://13.201.30.120:8080/"  
        TOMCAT_CREDENTIAL_ID = "tomcat"  
  
        // Slack  
        SLACK_CHANNEL = "jobs"
```

```
}
```

```
stages {
```

```
    stage('Stage-1: Clean Workspace') {
```

```
        steps {
```

```
            cleanWs()
```

```
        }
```

```
    }
```

```
    stage('Stage-2: Clone Source Code') {
```

```
        steps {
```

```
            git branch: 'main',
```

```
            url: 'https://github.com/betawins/hiring-app.git'
```

```
        }
```

```
    }
```

```
    stage('Stage-3: Validate Project') {
```

```
        steps {
```

```
            sh 'mvn validate'
```

```
        }
```

```
    }
```

```
    stage('Stage-4: Compile Application') {
```

```
steps {

    sh 'mvn compile'
}

stage('Stage-5: Run Unit Tests') {

    steps {

        sh 'mvn test'
    }
}

stage('Stage-6: SonarQube Analysis') {

    steps {

        withSonarQubeEnv('sonarqube') {

            sh """
            ${SCANNER_HOME}/bin/sonar-scanner \
            -Dsonar.projectKey=hiring-app \
            -Dsonar.projectName=hiring-app \
            -Dsonar.sources=.
            """
        }
    }
}

stage('Stage-7: Package WAR File') {
```

```
steps {

    sh 'mvn clean package'
}
}

stage('Stage-8: Archive Artifact') {

    steps {

        archiveArtifacts artifacts: 'target/hiring.war'
    }
}

stage('Stage-9: Upload Artifact to Nexus') {

    steps {

        nexusArtifactUploader(

            nexusVersion: NEXUS_VERSION,
            protocol: NEXUS_PROTOCOL,
            nexusUrl: NEXUS_URL,
            repository: NEXUS_REPOSITORY,
            credentialsId: NEXUS_CREDENTIAL_ID,
            groupId: GROUP_ID,
            version: VERSION,

            artifacts: [
                [
```

```
        artifactId: ARTIFACT_ID,  
        classifier: "",  
        file: 'target/hiring.war',  
        type: 'war'  
    ]  
]  
)  
}  
}
```

```
stage('Stage-10: Deploy WAR to Tomcat') {
```

```
    steps {
```

```
        deploy adapters: [
```

```
            tomcat9(  
                credentialsId: TOMCAT_CREDENTIAL_ID,  
                path: "",  
                url: "${TOMCAT_URL}"  
            )  
        ],
```

```
        contextPath: 'hiring-app',  
        war: 'target/hiring.war'
```

```
    }  
}
```

```
stage('Stage-11: Verify Deployment') {
```

```
    steps {
```

```
sh """"  
curl -I http://13.201.30.120:8080/hiring-app  
""""  
}  
}  
}
```

```
post {
```

```
    success {
```

```
        slackSend(  
            channel: "${SLACK_CHANNEL}",  
            color: 'good',  
            tokenCredentialId: 'slack',  
            message: """"
```

```
SUCCESS:
```

```
Multi-Stage Pipeline Completed Successfully
```

```
Job Name: ${env.JOB_NAME}
```

```
Build Number: ${env.BUILD_NUMBER}
```

```
Build URL: ${env.BUILD_URL}
```

```
""""
```

```
    )  
}
```

```
    failure {
```

```
        slackSend(  
            channel: "${SLACK_CHANNEL}",  
            color: 'danger',
```

```
        tokenCredentialId: 'slack',
        message: """"
```

**FAILED:**

**Multi-Stage Pipeline Failed**

**Job Name: \${env.JOB\_NAME}**

**Build Number: \${env.BUILD\_NUMBER}**

**Build URL: \${env.BUILD\_URL}**

```
""""
```

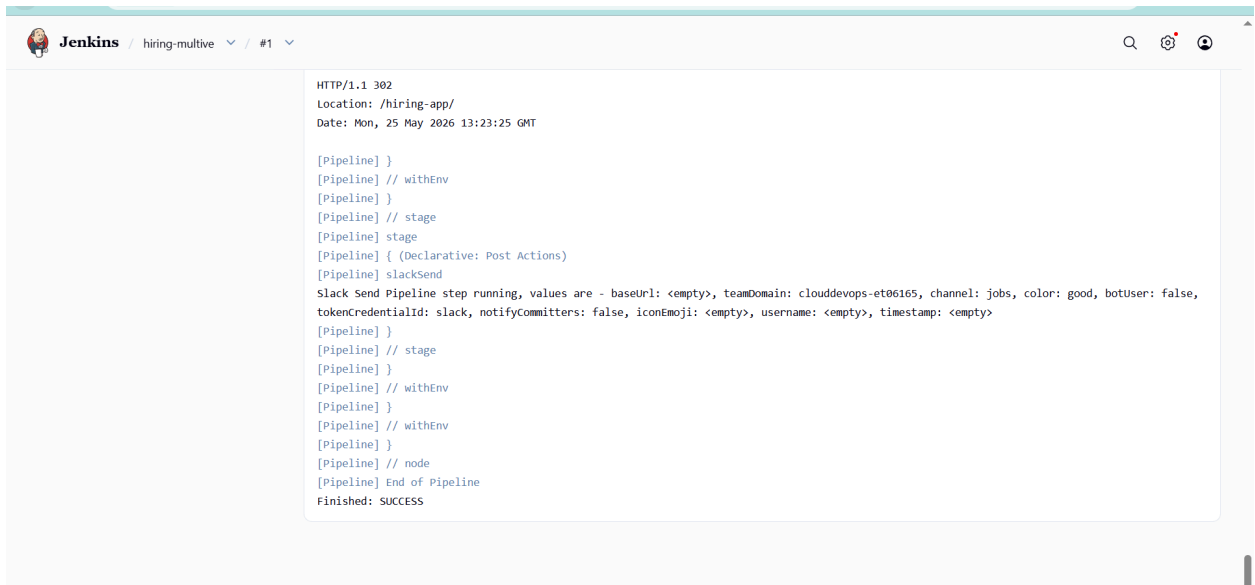
```
    )
  }
}
}
```

**save& build**

**Build successfull**

The screenshot displays the Jenkins web interface for a pipeline named 'hiring-multive'. The interface includes a left-hand navigation menu with options like 'Status', 'Changes', 'Build Now', 'Configure', 'Delete Pipeline', 'SonarQube', 'Stages', 'Rename', and 'Pipeline Syntax'. The main content area shows the pipeline's status as 'hiring-multive' with a green checkmark. Below this, it lists 'Last Successful Artifacts' including 'hiring.war' (1.66 KiB) and a 'view' link. A 'SonarQube Quality Gate' section indicates that the 'hiring-app' has 'Passed' and 'server-side processing' was a 'Success'. A 'Permalinks' section provides links to the last build, last stable build, last successful build, and last completed build, all of which occurred 1 minute and 22 seconds ago. At the bottom, a 'Builds' section shows a list of builds for 'Today', with build #1 at 13:23 marked as successful with a green circle icon.

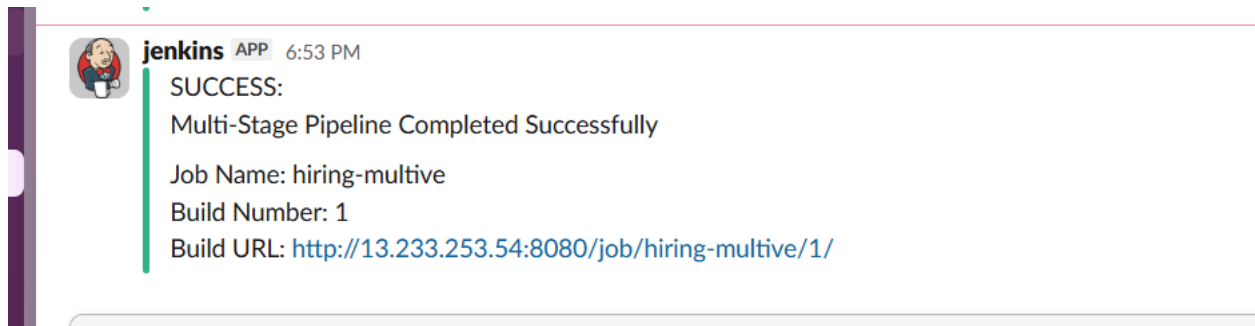
## Console output

A screenshot of the Jenkins web interface showing the console output for a job named 'hiring-multive', build #1. The output is a log of pipeline steps, including HTTP status, location, date, and various pipeline configuration steps like 'withEnv', 'stage', and 'slackSend'. The final status is 'Finished: SUCCESS'.

```
HTTP/1.1 302
Location: /hiring-app/
Date: Mon, 25 May 2026 13:23:25 GMT

[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] slackSend
slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: cloudevops-et06165, channel: jobs, color: good, botuser: false,
tokenCredentialId: slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

## Slack notification



## parallel stage pipeline job:

This pipeline runs multiple stages simultaneously to reduce build time.

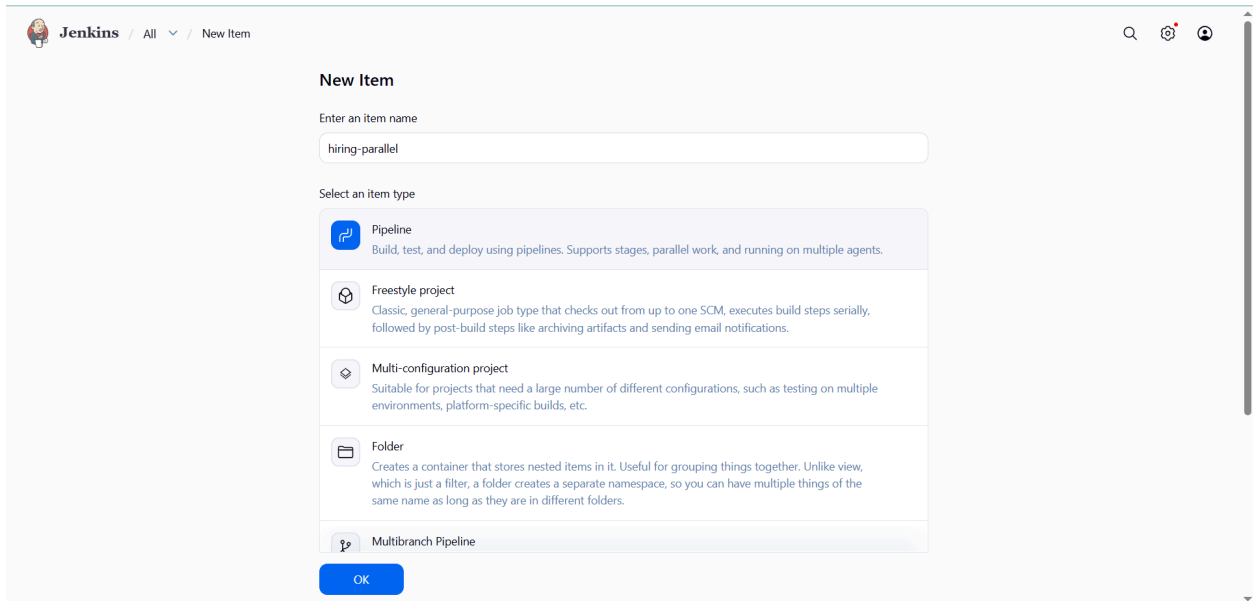
The following stages run together:

- Unit Tests
- SonarQube Analysis
- Security/Code Validation

This improves CI/CD performance.



## Create new item→select pipeline→configure



The screenshot shows the Jenkins 'New Item' configuration page. At the top, the breadcrumb is 'Jenkins / All / New Item'. The main heading is 'New Item'. Below it, there is a text input field labeled 'Enter an item name' with the value 'hiring-parallel'. Underneath, a section titled 'Select an item type' lists several options: 'Pipeline' (highlighted), 'Freestyle project', 'Multi-configuration project', 'Folder', and 'Multibranch Pipeline'. Each option has a brief description. At the bottom of the list is an 'OK' button.

**New Item**

Enter an item name

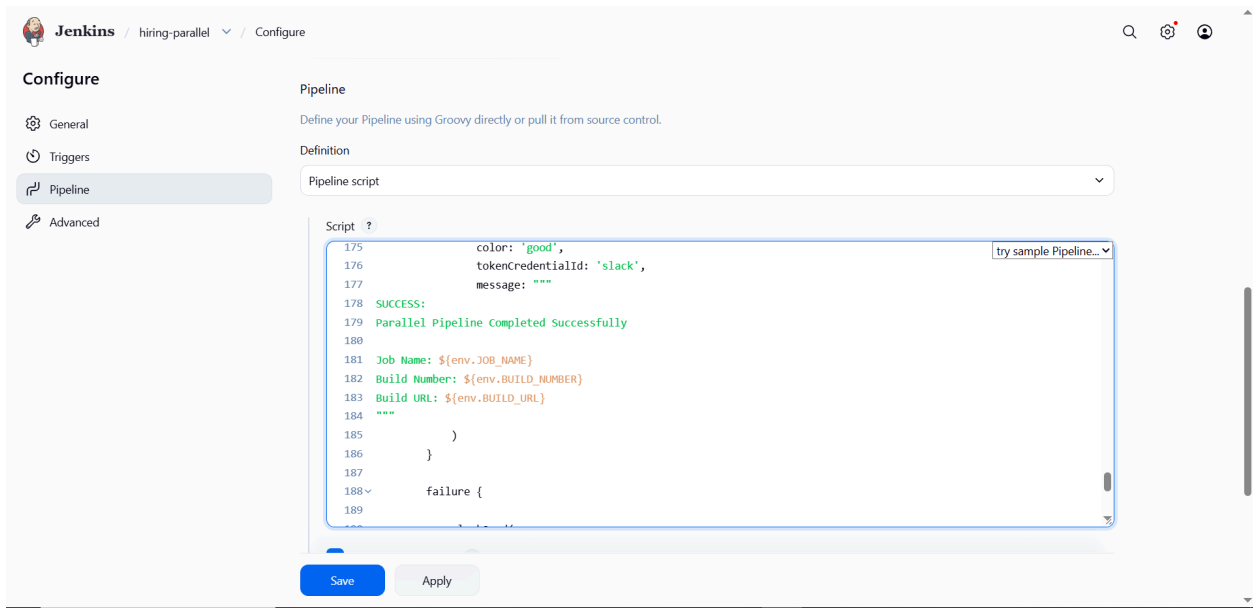
hiring-parallel

Select an item type

- Pipeline**  
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project  
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project  
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder  
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- Multibranch Pipeline

OK

## Write pipeline



The screenshot shows the Jenkins 'Configure' page for the 'hiring-parallel' item. The breadcrumb is 'Jenkins / hiring-parallel / Configure'. On the left, there is a sidebar with tabs: 'General', 'Triggers', 'Pipeline' (selected), and 'Advanced'. The main area is titled 'Pipeline' and contains the instruction 'Define your Pipeline using Groovy directly or pull it from source control.' Below this, there is a 'Definition' section with a dropdown menu set to 'Pipeline script'. A large text area labeled 'Script' contains a Groovy pipeline script. At the bottom, there are 'Save' and 'Apply' buttons.

**Configure**

- General
- Triggers
- Pipeline**
- Advanced

**Pipeline**

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script

```
175         color: 'good',
176         tokenCredentialId: 'slack',
177         message: ""
178     SUCCESS:
179     Parallel Pipeline Completed Successfully
180
181     Job Name: ${env.JOB_NAME}
182     Build Number: ${env.BUILD_NUMBER}
183     Build URL: ${env.BUILD_URL}
184     ""
185     )
186   }
187
188   failure {
189     
```

try sample Pipeline...

Save Apply

## save&build

 **Jenkins** / hiring-parallel

Status

</> Changes

▶ Build Now

⚙️ Configure

🗑️ Delete Pipeline

🔍 SonarQube

📁 Stages

✎ Rename

❓ Pipeline Syntax

✓ hiring-parallel

📦

Last Successful Artifacts

📄 hiring.war

1.66 KiB

👁️ view

**SonarQube Quality Gate**

hiring-app

Passed

server-side processing: 

Success

**Permalinks**

- [Last build \(#1\), 29 sec ago](#)
- [Last stable build \(#1\), 29 sec ago](#)
- [Last successful build \(#1\), 29 sec ago](#)
- [Last completed build \(#1\), 29 sec ago](#)

Builds >

🔍 Filter

/

Today

✓ #1 13:36

🔍 ⌵

## Console output

 **Jenkins** / hiring-parallel / #1

HTTP/1.1 302  
Location: /hiring-app/  
Date: Mon, 25 May 2026 13:37:03 GMT

[Pipeline] }  
[Pipeline] // withEnv  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] stage  
[Pipeline] { (Declarative: Post Actions)  
[Pipeline] slackSend  
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: cloudevops-et06165, channel: jobs, color: good, botUser: false, tokenCredentialId: slack, notifyCommiters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] }  
[Pipeline] // withEnv  
[Pipeline] }  
[Pipeline] // withEnv  
[Pipeline] }  
[Pipeline] // node  
[Pipeline] End of Pipeline  
Finished: SUCCESS

## Slack notification



**jenkins** APP 7:07 PM

SUCCESS:

Parallel Pipeline Completed Successfully

Job Name: hiring-parallel

Build Number: 1

Build URL: <http://13.233.253.54:8080/job/hiring-parallel/1/>

**B** *T* U